

# Full Stack Web Application

## Project Documentation Format

### 1. Introduction

**Project Title: FinWise AI - Intelligent Loan Eligibility, Credit Analysis & Financial Advisory Platform**

Team Members & Roles:

| Name                          | Role                                             |
|-------------------------------|--------------------------------------------------|
| Jainesh Bharti                | Frontend Development + AI (Groq) Integration     |
| Navneeth Biyyapu              | Frontend Development + Google Sheets Integration |
| Vishnu Vardhana Reddy Paluvai | Frontend Development + UI/UX                     |

### 2. Project Overview

**Purpose:** FinWise AI simplifies financial decision-making by letting users instantly check loan eligibility, analyze their credit score, calculate EMIs, and receive AI-generated personalised financial guidance - all in one responsive web app, with no signup required.

**Key Features:**

- Loan Eligibility Checker - rule-based validation against income, credit score, EMI and age thresholds
- Credit Score Analyzer - classifies profile into Excellent / Good / Poor with improvement tips
- EMI Calculator - standard reducing-balance EMI formula with amortisation summary
- AI Financial Tips - personalised recommendations generated via the Groq AI API
- Google Sheets Integration - every submission logged serverlessly via Google Apps Script
- Glassmorphism, mobile-responsive UI

### 3. Architecture

**Frontend:** Built with HTML5, CSS3 (glassmorphism styling, custom properties, responsive grid/flexbox) and vanilla JavaScript (ES6) - no framework is used, keeping the app lightweight and dependency-free.

**Backend:** There is no dedicated application server. Two lightweight services act as the backend: (1) the Groq AI Chat Completions API, called directly from the browser for financial-tip generation, and (2) a Google Apps Script Web App that exposes a doPost endpoint.

**Database:** Google Sheets is used as a serverless data store. Each user submission (inputs + computed result) is appended as a new row via the Apps Script webhook - there is no relational/NoSQL database.

### 4. Setup Instructions

**Prerequisites:** A modern web browser; a free Groq API key (from console.groq.com); optionally, a Google account to deploy your own Apps Script + Sheet.

**Installation:**

- Clone the repository: `git clone https://github.com/JaineshBharti5/finwise-ai.git`
- Open the folder and launch index.html directly in a browser, or serve it with any static server
- Enter your Groq API key under 'API Settings' in the sidebar to enable AI Financial Tips
- (Optional) Deploy google-apps-script.gs as a Web App in Google Apps Script and update the endpoint URL in the JS files to enable Sheets logging

## 5. Folder Structure

| Path                                   | Purpose                                                     |
|----------------------------------------|-------------------------------------------------------------|
| index.html                             | Landing page / dashboard                                    |
| eligibility.html + js/eligibility.js   | Loan Eligibility Checker page and logic                     |
| credit-score.html + js/credit-score.js | Credit Score Analyzer page and logic                        |
| emi.html + js/emi.js                   | EMI Calculator page and logic                               |
| tips.html + js/tips.js                 | AI Financial Tips page and Groq API integration             |
| css/style.css                          | Shared glassmorphism stylesheet                             |
| js/app.js                              | Shared app-wide JavaScript (navigation, shared UI logic)    |
| google-apps-script.gs                  | Serverless backend script for logging data to Google Sheets |

## 6. Running the Application

As a fully static site, FinWise AI needs no build step or server process:

- Locally: open index.html directly in a browser, or run a simple static server (e.g. `npx serve .`)
- Production: served automatically via GitHub Pages at <https://jaineshbharti5.github.io/finwise-ai>

## 7. API Documentation

| Endpoint                                                          | Method | Parameters                                                                | Example Response                                                  |
|-------------------------------------------------------------------|--------|---------------------------------------------------------------------------|-------------------------------------------------------------------|
| Groq AI Chat Completions<br>(api.groq.com)                        | POST   | model, messages[] (user's financial profile as prompt), API key in header | JSON with an AI-generated financial-tips message                  |
| Google Apps Script Web App<br>(script.google.com/macros/.../exec) | POST   | JSON body: name, salary, credit score, EMI, age, result                   | JSON acknowledgement; new row appended to the linked Google Sheet |

## 8. Authentication

FinWise AI does not implement user login/authentication - it is a stateless, no-signup tool. The only credential involved is the user-supplied Groq API key, entered client-side under API Settings and used solely to authorize AI Tips requests directly from the browser to Groq.

## 9. User Interface

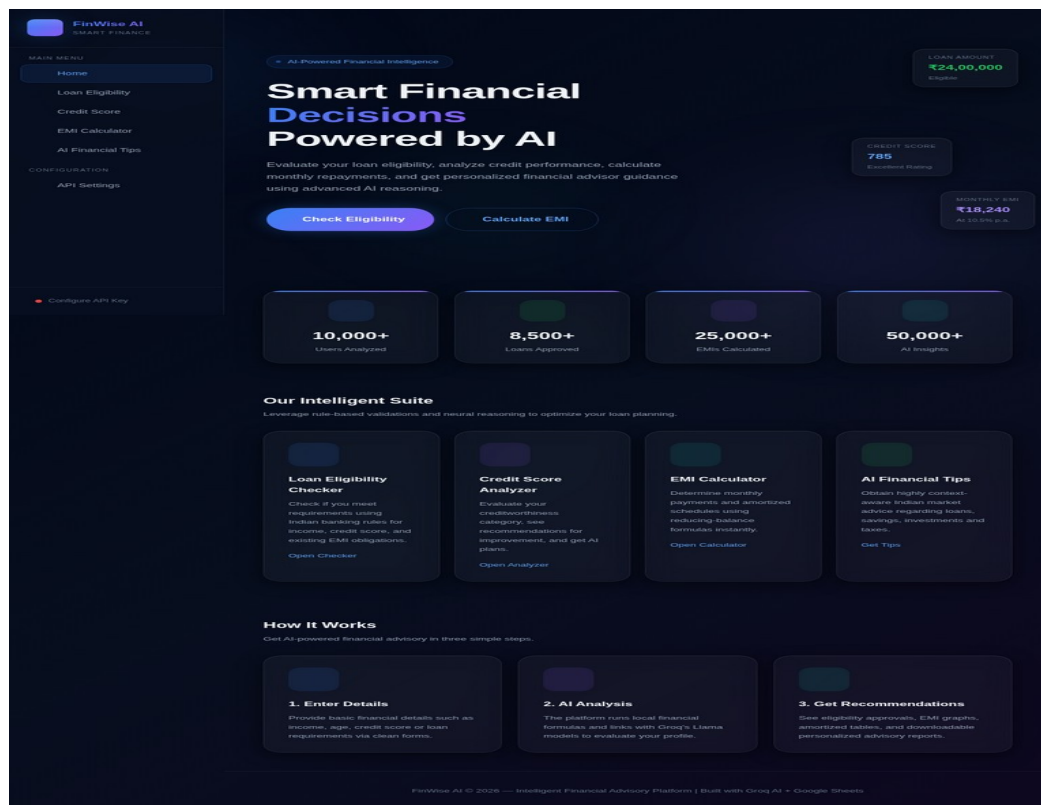


Figure: FinWise AI landing dashboard.

## 10. Testing

Testing combined manual functional testing of each rule engine (eligibility, credit score, EMI) against hand-calculated expected values, functional/performance testing of the Groq AI integration (response correctness and latency), and User Acceptance Testing (UAT) covering end-to-end scenarios across devices. See the Project Development Phase documents for full test case tables.

## 11. Screenshots / Demo

Live demo: <https://jaineshbharti5.github.io/finwise-ai>

Demo video: <https://drive.google.com/file/d/1R7yStCQXAuEFCktQstJzgT8etC3Lp7Fr/view?usp=sharing>

GitHub repository: <https://github.com/JaineshBharti5/finwise-ai>

## 12. Known Issues

- AI Financial Tips requires the user to supply their own free Groq API key; there is no shared/managed key.
- Google Sheets logging depends on the Apps Script Web App URL being correctly deployed and kept up to date in the frontend.
- No server-side validation - all checks are client-side, so the tool is best used for self-assessment, not as a system of record.

## 13. Future Enhancements

- User accounts to track eligibility/credit history over time

- A lightweight backend (Node.js/Express or serverless functions) to secure the Groq API key server-side
- Downloadable PDF summary of eligibility, credit and EMI results
- Multi-language support for wider accessibility
- Chatbot-style conversational interface for financial Q&A